

High-Level Synthesis for Scheduling Pipelined Circuits

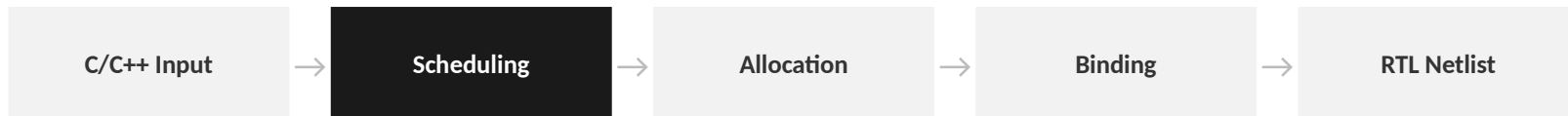
Md Ratul Ahmed Alvi

HW/SW Codesign Seminar • Group B5 • SS2026
Hochschule Hamm-Lippstadt (HSHL)
June 22, 2026

Main reference: Teich / de Micheli

Introduction to High-Level Synthesis

High-Level Synthesis converts C/C++ behavioral descriptions into RTL hardware implementations.



Scheduling assigns each operation to a clock cycle.
It determines throughput, latency, and resource usage of the final hardware.

This paper focuses on pipeline scheduling — overlapping loop iterations to increase throughput.

Pipelining in HLS

Pipelining overlaps consecutive loop iterations so that while one iteration is still processing, the next has already started.

Latency	Total clock cycles for one iteration (input to output)
Throughput	Results produced per clock cycle = $1 / II$
Initiation Interval (II)	Clock cycles between two consecutive iteration starts
ResII	Resource limitation: $\lceil \text{operations} \div \text{hardware units} \rceil$
RecII	Dependency limitation: $\lceil \text{loop delay} \div \text{iteration distance} \rceil$

$$II_{\min} = \max(ResII , RecII)$$

Problem Statement

Why cannot all designs achieve $\Pi = 1$?

Two independent factors limit the achievable initiation interval.

ResII — Resource constraint

Limited hardware forces operations to take turns. Can be improved by adding more hardware units.

RecII — Dependency constraint

Loop-carried dependencies chain iterations together. Cannot be improved by adding hardware — must change the algorithm.

Three experiments isolate each constraint:

FIR Filter (resource-limited) · Accumulation Loop (dependency-limited) · Delay Line (no limit)

Experiment 1 — FIR Filter

$y = a \cdot x[0] + b \cdot x[1] + c \cdot x[2]$ | 1 pipelined multiplier, 2 adders



Schedule:

	0	1	2	3	4
Mult	M1	M2	M3	—	—
Add	—	—	—	A1	A2

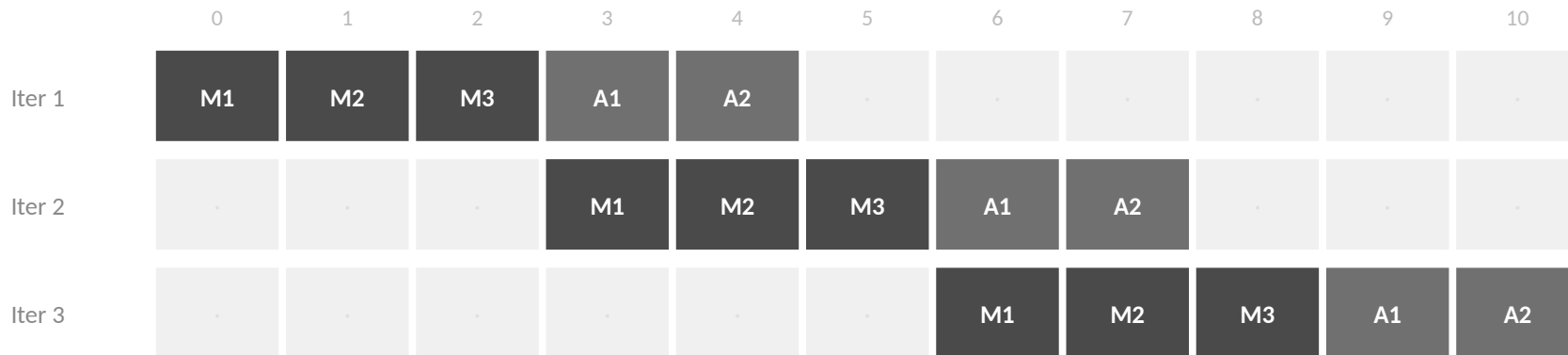
ResII = $\lceil 3/1 \rceil = 3$ No loop-carried dependency $\rightarrow$ II = 3

3 multiplications, 1 pipelined multiplier. Multiplier accepts new input every cycle.

M1 enters cycle 0, M2 cycle 1, M3 cycle 2. Three cycles to feed all jobs. Speedup = 2.67×

Experiment 1 — Pipeline (II = 3)

3 iterations overlapping. Each starts 3 cycles after the previous.



At cycles 3-4: Iter 1 uses adder (A1, A2) while Iter 2 uses multiplier (M1, M2).
Different resources, same clock cycles — no conflict. That is pipelining.

Experiment 2 — Accumulation Loop

$\text{sum} += x[i] \times \text{coeff}$ | 1 multiplier, 1 adder | Loop-carried dependency



Schedule:

	0	1	2
Mult	M1 starts	M1 busy	free
Add	wait	wait	A1

$$\text{ResII} = 1 \quad \text{RecII} = 2 \quad \rightarrow \quad \text{II} = \max(1, 2) = 2$$

M1 takes 2 cycles. A1 cannot start until cycle 2. The scheduler tried $\text{II} = 1$ but operation latency made it infeasible. First feasible schedule: $\text{II} = 2$.

Even with extra adders, ResII would drop but II stays 2 because the loop-carried dependency fixes the recurrence limit. Speedup = 1.50×

Experiment 2 — Pipeline (II = 2)

4 iterations overlapping. Notice the gaps compared to FIR.



FIR has dense cells. This grid has "M1 busy" and grey gaps — the dependency forces iterations apart. More hardware would not change this.

Experiment 3 — Delay Line

$y[n] = x[n] \times \text{coeff}$ | 2 multipliers, no dependencies



$\text{ResII} = \lceil 1/2 \rceil = 1$ No loop-carried dependency $\rightarrow \text{II} = 1$

One multiply per iteration, two multipliers. They alternate: #1, #2, #1, #2.
A new iteration starts every single cycle. Speedup = 2.00×

Latency: each multiply still takes 2 cycles (unchanged).
Throughput: one result per cycle (improved). That is pipelining.

Experiment 3 — Pipeline (II = 1)

5 iterations overlapping. Two multipliers alternate.

	0	1	2	3	4	5	6
Iter 1	M1 #1	M1 #1 busy	done	.	.	.	.
Iter 2	.	M1 #2	M1 #2 busy	done	.	.	.
Iter 3	.	.	M1 #1	M1 #1 busy	done	.	.
Iter 4	.	.	.	M1 #2	M1 #2 busy	done	.
Iter 5	.	.	.	.	M1 #1	M1 #1 busy	done

Multipliers alternate: #1 → #2 → #1 → #2. From cycle 2 onward, one result per cycle.

C++ Scheduler — Terminal Output

Compiled and tested on macOS using g++ -std=c++17

```
==== Experiment 1: 3-tap FIR filter (1 multiplier) ====
ResII (resource limit)      = 3
RecII (recurrence limit)    = - (no loop-carried dependency)
II = max(ResII,RecII)       = 3 -> throughput 1/3 per cycle
schedule (latency = 4):
  m0    step 0  MUL  slot 0
  m1    step 1  MUL  slot 1
  m2    step 2  MUL  slot 2
  a0    step 2  ADD  slot 2
  a1    step 3  ADD  slot 0

==== Experiment 2: accumulation loop (loop-carried dependency) ====
ResII (resource limit)      = 1
RecII (recurrence limit)    = 2
II = max(ResII,RecII)       = 2 -> throughput 1/2 per cycle
note: even with extra adders ResII would drop but II stays 2,
      because the loop-carried dependency fixes the recurrence limit.

==== Experiment 3: 2-tap delay line (2 multipliers) ====
ResII (resource limit)      = 1
RecII (recurrence limit)    = - (no loop-carried dependency)
II = max(ResII,RecII)       = 1 -> throughput 1/1 per cycle
```

Comparison of All Experiments

Experiment	Bottleneck	II	Speedup	Improvement
FIR Filter	Resources	3	2.67×	Add more multipliers
Accumulation	Dependencies	2	1.50×	Change the algorithm
Delay Line	None	1	2.00×	Already optimal

Conclusion

$$II_min = \max(ResII, RecII)$$

Pipelining improves throughput by overlapping loop iterations

FIR filter — limited by resources ($ResII = 3, II = 3$)

Accumulation — limited by dependencies ($RecII = 2, II = 2$)

Delay line — no bottleneck, ideal behavior ($II = 1$)

Throughput improves — latency does not change

Implementation: C++17 list scheduler, tested on three examples

References

- [1] G. de Micheli, Synthesis and Optimization of Digital Circuits, McGraw-Hill, New York, 1994.

- [2] J. Teich, Digitale Hardware/Software-Systeme: Synthese und Optimierung, Springer, 2nd edition, 2007.

- [3] A. M. Sllame and V. Drabek, "A pipeline scheduling algorithm for high-level synthesis," IFAC Proceedings Volumes, vol. 36, no. 14, pp. 19–24, 2003.

- [4] Z. Zhang and B. Liu, "SDC-based modulo scheduling for pipeline synthesis," in Proc. IEEE/ACM ICCAD, 2013, pp. 211–218.

- [5] J. Oppermann, M. Kumm, O. Reiche, J. Teich, and P. Zipf, "Exact and practical modulo scheduling for high-level synthesis," ACM Trans. on Reconfigurable Technology and Systems, vol. 12, no. 4, pp. 1–26, 2019.

Questions?

Thank you for your attention.

Md Ratul Ahmed Alvi · Group B5 · HW/SW Codesign · SS2026